

Project Progress Summary

Group Members

- | | |
|------------------------------------|---|
| 1. Moharnab Gogoi (Intern ID: 1) | <i>IoT, Backend & Cloud Integration</i> |
| 2. Aryyaman Bora (Intern ID: 38) | <i>Frontend & UI</i> |
| 3. Mayuree Khanikar (Intern ID: 2) | <i>Documentation & Research</i> |
| 4. Indrani Gogoi (Intern ID: 96) | <i>Documentation & Research</i> |

Supervisor Name

Ashish Kumar Mahato

Project Title

FloodEye: Real-Time Flood Alert & Water Level Monitoring System

Abstract

FloodEye is a highly responsive, AI-powered Progressive Web Application (PWA) designed for real-time flood monitoring and early warning. The system collects environmental telemetry—water level (distance), temperature, humidity, atmospheric pressure, and altitude—from an ESP32 microcontroller equipped with a DHT11, BMP180, and HC-SR04 sensor array. Readings are uploaded every 15 seconds to the ThingSpeak IoT cloud platform, from where a Next.js 16 full-stack backend fetches, caches, and serves the data to a React dashboard.

A key differentiator of FloodEye is its in-browser machine learning engine: a hybrid **1D-CNN + Bidirectional LSTM + Attention** deep learning model (trained on Assam flood-sensor datasets) that runs entirely client-side via TensorFlow.js, producing 4-hour-ahead water level predictions with risk classification (Normal / High / Critical). The dashboard features a glassmorphism UI, live MapLibre GL sensor map, threshold-based Smart Alerts Engine, environmental comfort score, historical analytics with selectable time windows, and one-click data export (CSV/PDF). The app is installable as a PWA on iOS, Android, and desktop, making it accessible to field workers and government officials without platform constraints.

Details of the Software/Hardware to be Implemented

1. Off-Screen Push & Email Notifications

Implement Web Push Notifications via the browser Push API and Service Workers so that critical flood alerts (e.g., “Water Level Critical”, “Sensor Offline”) are delivered even when the dashboard tab is closed. Email alerts will be dispatched through the **Resend** or **SendGrid** transactional email API from a Next.js server action.

2. Multi-Node Fleet Management

Extend the database schema and dashboard UI to manage **multiple ESP32 devices** across different geographic locations. Each device will appear as an interactive pin on the MapLibre map; clicking a pin will load device-specific telemetry and ML predictions.

3. External Weather API Correlation

Integrate the **OpenWeatherMap API** to correlate localized ESP32 sensor data (temperature, humidity, pressure) with regional precipitation forecasts, improving the contextual accuracy and interpretability of flood risk assessments.

4. Generative AI “Environmental Summaries”

Pass the last 24 hours of telemetry and TensorFlow.js model output to the **Google Gemini API** to generate

natural-language summaries (e.g., “*Water levels have risen 12% in the last 3 hours. The AI predicts high flood risk by 4:00 PM.*”), making the dashboard accessible to non-technical government officials and citizens.

5. CSV Data Export

Add a CSV Export button in the Historical Analytics section so researchers and data scientists can download raw time-series sensor data for offline analysis in Python or Excel.

6. MQTT Real-Time Streaming

Replace or supplement the 15-second HTTP polling interval with **MQTT over WebSockets** (broker: HiveMQ Cloud or AWS IoT Core) so the dashboard updates the instant the ESP32 publishes a reading, eliminating polling latency.

7. Interactive Sensor Calibration UI

Add a “Calibration” panel in the Settings page where users input the **baseline distance** from the HC-SR04 to the riverbed, enabling accurate water depth computation: $Water\ Depth = Baseline\ Distance - Current\ Distance$.

Components Required (if any)

Hardware components

- **ESP32 DevKit boards** — one per monitoring location for multi-node fleet management.
- **HC-SR04 Ultrasonic Sensors** — one per additional node.
- **DHT11 / DHT22 Sensors** — temperature and humidity per node.
- **BMP180 / BMP280 Sensors** — pressure and altitude per node.
- **Waterproof enclosures** — for outdoor/riverbank deployment of sensor nodes.
- **Power supply:** Solar panel + LiPo battery + TP4056 charging module for field-deployed, off-grid nodes.
- **Jumper wires, breadboard, resistors** — prototyping and connections.

Software / API Dependencies

- **web-push** (npm) — VAPID key generation and push notification subscription management.
- **Resend or SendGrid SDK** — transactional email API for alert emails.
- **OpenWeatherMap API key** — free-tier weather data for correlation queries.
- **@google/genai SDK + Gemini API key** — Generative AI environmental summaries.
- **mqtt** (npm) — MQTT over WebSocket client for Next.js.
- **HiveMQ Cloud / AWS IoT Core** — cloud MQTT broker endpoint.
- **papaparse** (npm) — client-side CSV generation and download.
- **Additional ThingSpeak channels** — one per ESP32 node for multi-device data separation.

Cloud / Infrastructure

- **Vercel** — existing hosting; Pro plan may be needed for long-running MQTT WebSocket connections.
- **Neon PostgreSQL** — existing database; schema extension for multi- device records, push subscriptions, and calibration settings.